补充材料：完整证明

《状态不是历史，源才是：过程同一性、责任承接与最小修订》

高健（Jian Gao）

本补充给出正文 §6.5、§7.3 末、§8.5、§9 各结果及 §10.1 原 comb ／ CPS1 实装的完整书面证明。正文已陈述结果并给出证明思路；这里逐条交代对象与定义域、实际输入与量词、关键引理、决定性推导，以及同一对象在后续结果中的直接使用，最后对应到形式证明。记号与正文相同，定义、定理与公式编号均指正文。正文 §2–§5、§6.1–§6.4、§7.1–§7.3 与 §8.1–§8.4 的证明已在正文与附录 A–C 中完整给出，不再重复。

每节末的“形式对应”列出承载该结果的 Lean 声明；所在源提交与完整路径见正文附录 D.2–D.3，公开复现入口见正文“代码与数据可得性”。

本补稿的固定来源：PR = `e05558097256e86b537c019c8ca63cd449d6b2d2`，MF = `c62f3d25c42d4bf2b77ac009b49fc71cfd752f46`，MV2 = `71e94e8261d67cf4849a05eb14ac95aa2f7dcbb9`，MV3 = `9c73a630ce05bea062ec889b188f5105c0afb796`（Homework）；H0-S = `a1adcc9f4a06b872c381b8ad49513d6bf1cd621a`（公开源配置）。附录 D.3 给出完整路径缩写及声明入口。

S0 共同工具

S0.1 剩余计数与执行。命题 3.7 给出：每个执行步恰使剩余计数减 1；常量没有执行步；从左到右的执行 $\mathrm{exec}_\rho(e) : e \to \mathrm{const}(\mathrm{ev}_\rho e)$ 的长度为 $r(e)$；每条执行迹都是推导，故端点求值相等（命题 3.6(1)，下称**可靠性**）。

S0.2 配对语言保持剩余计数。附录 A.2 的配对化 $e \mapsto \widetilde{e}$ 把变量、常量、加法、线性与双线性节点逐一换成配对值上的同类节点，因此 $r(\widetilde{e}) = r(e)$（对 e 的结构归纳，每个节点贡献的计数不变），且

$$\mathrm{ev}_{(\rho,\varepsilon)}(\widetilde{e}) = (\mathrm{ev}_\rho(e),\ \mathrm{eff}_{\rho,\varepsilon}(e)). \tag{S0.1}$$

S0.3 迹的关系像与更新读法。设 $t : e \to e'$ 是环境 ρ 下的执行迹。每个执行步是一枚推导证据，其关系像是两端点之差；整条迹的关系像伸缩为 $e - e'$。由可靠性，$e - e' \in \ker(\mathrm{ev}_\rho)$。给定增量 ε，命题 3.3 的更新等式给出

$$\mathrm{ev}_{\rho+\varepsilon}(e - e') = \mathrm{ev}_\rho(e - e') + \mathrm{eff}_{\rho,\varepsilon}(e - e') = \mathrm{eff}_{\rho,\varepsilon}(e) - \mathrm{eff}_{\rho,\varepsilon}(e'). \tag{S0.2}$$

特别地，$e' = \mathrm{const}(c)$ 时 $\mathrm{eff}(e') = 0$，更新后的值就是 $\mathrm{eff}_{\rho,\varepsilon}(e)$。

S0.4 余像与诱导映射。对 $\mathbb{Z}$-线性映射 $f : X \to Y$，记 $\mathrm{Res}(f) := X/\ker f$ 为其余像，$\mathrm{can}_f : X \to \mathrm{Res}(f)$ 为商映射；$\mathrm{Res}(f) \cong \mathrm{range}(f)$。若 $f' \circ s = t \circ f$（称 (s,t) 为从 f 到 f' 的态射），则 $s(\ker f) \subseteq \ker f'$，故 s 诱导 $\overline{s} : \mathrm{Res}(f) \to \mathrm{Res}(f')$，满足 $\overline{s} \circ \mathrm{can}_f = \mathrm{can}_{f'} \circ s$。$\mathrm{can}_f(x) = 0$ 当且仅当 $f(x) = 0$。

S0.5 提升残差。设 (s,t) 是从 $f:C\to D$ 到 $f':C'\to D'$ 的态射。对 $c\in C$，其**纤维** $\mathrm{Fib}_f(c):=\{x:f(x)=f(c)\}$。$s$ 把 $\mathrm{Fib}_f(c)$ 送入 $\mathrm{Fib}_{f'}(sc)$，并把 $\ker f$ 送入 $\ker f'$。对 $y\in\mathrm{Fib}_{f'}(sc)$，令

$$\mathrm{lift}(c,y):=[\,y-s(c)\,]\in\ker f'\big/s(\ker f).$$

引理 S0.5. $\mathrm{lift}(c,y)=0$ 当且仅当存在 $x\in\mathrm{Fib}_f(c)$ 使 $s(x)=y$。

证明。若 $y-s(c)=s(k)$，$k\in\ker f$，取 $x=c+k$，则 $f(x)=f(c)$ 且 $s(x)=y$。反之若 $x\in\mathrm{Fib}_f(c)$ 且 $s(x)=y$，则 $k=x-c\in\ker f$，$y-s(c)=s(k)$。■

S0.6 作用词模型。设 X 为加法群，$(A_\ell)_{\ell\in L}$ 为其上一族加性作用，$\mathrm{rd}:X\to Y$ 为加性读出。对词 $w=\ell_1\cdots\ell_n$（先作用 ℓ_1）记 $A_w:=A_{\ell_n}\cdots A_{\ell_1}$。令 $K:=\{x:\forall w,\ \mathrm{rd}(A_wx)=0\}$。

引理 S0.6. (i) K 对每个 A_ℓ 封闭，因而每个 A_ℓ 诱导到 $\mathrm{Mod}:=X/K$；(ii) $[x]=[y]$ 当且仅当 $\forall w,\ \mathrm{rd}(A_wx)=\mathrm{rd}(A_wy)$；(iii) 把 $[x]$ 送到读出族 $(w\mapsto\mathrm{rd}(A_wx))$ 的映射单射，因而 Mod 与这一族的像（即全部词读出的完成载体）线性同构，同构与各作用交换。

证明。(i) 若 $x\in K$，则对任意 w，$\mathrm{rd}(A_wA_\ell x)=\mathrm{rd}(A_{\ell w}x)=0$。(ii) 由 K 的定义与线性。(iii) 由 (ii)，交换性在代表元上逐词验证。■

S0.7 效应历史方程。设另有 W 上的加性作用 (U_ℓ) 与加性观察 $M:X\to W$。对点 x 与词 w，令总残差 $R(w,x):=U_wM(x)-M(A_wx)$；单字母残差 $r(\ell,x):=U_\ell M(x)-M(A_\ell x)$；递归定义

$$\mathrm{Acc}([\,],x)=0,\qquad \mathrm{Acc}(\ell w,x)=U_w\,r(\ell,x)+\mathrm{Acc}(w,A_\ell x).$$

引理 S0.7. $\mathrm{Acc}(w,x)=R(w,x)$；记录 $(\ell_k,x_{k-1},r(\ell_k,x_{k-1}))$ 的历史长度为 $|w|$。

证明。对 w 的长度归纳：

$$\begin{aligned}R(\ell w,x)&=U_wU_\ell M(x)-M(A_wA_\ell x)\\&=U_w(U_\ell M(x)-M(A_\ell x))+(U_wM(A_\ell x)-M(A_wA_\ell x)),\end{aligned}$$

第一项是 $U_w\,r(\ell,x)$，第二项是 $R(w,A_\ell x)$。■

S0.8 联合作用词载体。设 I,H,Q 为加法群，带三族作用 $(\alpha_\ell),(\eta_\ell),(\mu_\ell)$ 与两个加性读出 $c:I\to H$、$m:I\to Q$。在 $I\times H\times Q$ 上令 $\theta_\ell:=\alpha_\ell\times\eta_\ell\times\mu_\ell$，种子 $\mathrm{seed}(i):=(i,c(i),m(i))$。**完整轨道载体** $\mathcal{C}$ 是全部 $\theta_w(\mathrm{seed}\,i)$ 张成的子模。

引理 S0.8. (i) $\mathcal{C}$ 是包含全部种子且对每个 θ_ℓ 封闭的最小子模；(ii) 三个分量投影 π_I,π_H,π_Q 与词作用交换；(iii) 关联元 $\mathrm{inc}(w,i):=\theta_w(\mathrm{seed}\,i)-\mathrm{seed}(\alpha_wi)$ 的 I 分量为零，H、Q 分量分别是 $\eta_wc(i)-c(\alpha_wi)$ 与 $\mu_wm(i)-m(\alpha_wi)$。

证明。(i) 生成子模由定义包含种子；任一包含种子且封闭的子模包含全部 $\theta_w(\mathrm{seed}\,i)$，故包含其张成。$\mathcal{C}$ 自身对 θ_ℓ 封闭，因为生成元的像仍是生成元。(ii) θ_ℓ 是分量积。(iii) 由 (ii) 逐分量计算。■

S1 定理 6.10：无旧承担者的责任出生与付款

对象与定义域。固定世界网络 N、其上一个带完整整账根登记的权威根、原点当前 c，以及读取器：它对根在 c 处的每个发生给出完整的环境—表达式对。记 o 为 c 处的规范发生，(ρ,e) 为读取器在 o 的输出，$B=r(e)$，$\Lambda=\Lambda_{\rho,e}$ 为定义 6.9 的执行债律。

输入与量词。定理对上述任意数据成立。不输入旧责任行、已登记的询问、程序表、未来请求表或空覆盖证书；原点支撑在整个计算中固定。

构造。

1. 扩世界与词汇。扩世界 $N[\Lambda]$ 由定义 6.3 给出：支撑为 $\mathrm{Sup} \times \mathrm{Option}$(债状态)。词汇的当前取 Λ 的债状态，即对 (e', t)，$t : e \to e'$ 为已付执行迹；初始当前为 (e, id)。当前 x 的支撑是 $(\mathrm{supp}(o), \mathrm{some}\ x)$。
2. 事件代数。在当前 $x = (e', t)$ 上，**源选动作**是执行器的输出：若 e' 为常量，取结算；否则取命题 3.7 执行的第一步 $s : e' \to e''$。后继当前 x^+ 在结算处为 x，在执行步处为 $(e'', t \cdot s)$。编译把结算送到 Λ 的恒同输运（定义 6.9 第四项），把执行步送到原生写入。
3. 整账演化。$\mathrm{whole}(x)$ 把支撑 $(\mathrm{supp}(o), x)$ 上的每一行送到 $(\mathrm{supp}(o), x^+)$ 上：数学行 $\mathrm{row}(x)$（Λ 在状态 x 的债行）送到 $\mathrm{row}(x^+)$；每条旧行原样输运。去向与起源互逆。
4. 补丁、余项与证书。有限补丁只含一条生成行，即数学行；其余全部行由同源输运余项覆盖——余项的发生是“该行属于旧账”的证据，无需旧账有限。补丁折叠回整账演化：$\mathrm{fold}(\mathrm{patch}(x)) = \mathrm{whole}(x)$。重构证书由整账演化去向的单射性生成。
5. 编译器、根与过程。编译器在每个当前发射补丁与整账；整账根由源与编译器组成；权威根与活过程按根登记的标准构造得到。过程在计数 k 的状态记为 x_k，满足 $x_0 = (e, \mathrm{id})$ 与 $x_{k+1} = x_k^+$。

关键引理。

引理 S1.1（一步预算）. $r(x^+) = r(x) - 1$（自然数减法）；若 $r(x) = 0$，则 $x^+ = x$。

证明。$r(x) > 0$ 时 e' 非常量，源选动作是执行步，由 S0.1 剩余计数减一；$r(x) = 0$ 时 e' 为常量，动作是结算，后继不变。■

引理 S1.2（预算与历史）. 对每个 k：$r(x_k) = B - k$，且 $|t_k| + r(x_k) = B$，其中 t_k 为 x_k 的已付迹。

证明。对 k 归纳。$k = 0$ 时 t_0 为空、$r(x_0) = B$。归纳步：若 $r(x_k) > 0$，执行步使迹长加一、计数减一；若 $r(x_k) = 0$，状态不变，且 $B - (k+1) = 0$。■

决定性推导。

1. 出生。由构造 2–4，每个当前恰发射一个事件；编译在结算处为恒同输运，在执行步处为原生写入。补丁恰含一条生成行；全部旧行作为余项保留，起源—去向互逆给出重构证书。原支撑不变（支撑第一分量恒为 $\mathrm{supp}(o)$），法则面、旧投影与原点处的类型与值观察作为根读出保留。
2. 付款。第 k 访问的同债当前是 $\mathrm{row}(x_k)$；去向等式 $\mathrm{whole}(x_k)(\mathrm{row}(x_k)) = \mathrm{row}(x_{k+1})$ 沿 k 归纳给出它到 $\mathrm{row}(x_0)$ 的谱系。其预算为 $r(x_k) = B - k$（引理 S1.2）。$k < B$ 时 $r(x_k) > 0$，源选动作是执行步 s，同债目标是 $\mathrm{row}(x_{k+1})$，预算严格下降（引理 S1.1）；以两侧空历史合成为定义 6.6 的付款宏。目标预算 $\leq$ 当前预算，故不回充；续行关系沿预算严格下降，良基。
3. 结算。$k = B$ 时 $r(x_B) = 0$，故 x_B 的表达式为常量 $\mathrm{const}(v)$，局部结算存在；预算为零的当前没有付款宏。已付迹 $t_B : e \to \mathrm{const}(v)$ 的长度为 B（引理 S1.2），由可靠性 $v = \mathrm{ev}_\rho(e)$。由引理 S1.1，$x_{B+1} = x_B$；访问计数继续推进，因果深度不同的访问在有限历史中仍被区分。
4. 收据。由 S0.3，t_B 的关系像是 $e - \mathrm{const}(\mathrm{ev}_\rho e) \in \ker(\mathrm{ev}_\rho)$；对任意增量 ε，它在 $\rho + \varepsilon$ 下的值是 $\mathrm{eff}_{\rho,\varepsilon}(e)$。■

直接使用。命题 6.11 以整树程序为读取器调用本定理；定理 7.7 与 §9 的各个运行都以同一所有者无关安装为数学行，读取其值、计费历史、原材料与关系边界。

形式对应。`JS/OwnerFree/Source.lean`（`vocabulary`、`eventAlgebra`、`whole`、`mathEntry`、`destination_math`）；`OwnerFree/Compiler.lean`（`remainder`、`patch`、`patch_fold`、`compiler`、`ledgerRoot`）；`OwnerFree/Completion.lean`（`next_budget`、`budget`、`history_accounting`、`completed_value`、`completed_history_length`、`completed_next_state`）；`OwnerFree/Payment.lean`（`lineage`、`debtCurrent`、`activePayment`、

`wellFounded`、`no_refill`、`endpoint_no_paid`）；`OwnerFree/Consumer.lean`（`value_source`、`paid_history`、`relation_boundary`、`updated_inverse_fibre`）。提交 AB。

S1.3 定理 6.12：Type 完成回执与持续出生

数据与构造。固定原程序 $\mathcal{P}$、递归帧 Φ_i 和实际动作 a_i；B_i 为当前已付状态表达式的剩余计数。回执的定义域是实际起点 i，返回的不是命题存在量词，而是依赖和

$$\sum_{d:\mathbb{N}} \{s : \mathsf{Settlement}(\Phi_{i+d}.\mathrm{state}) \mid d \le B_i \ \wedge \ a_{i+d} = \mathrm{inl}\, s\}.$$

按 B_i 强递归消去 a_i。结算支选 $d = 0$ 与实际 s；付款支选 $a_i = \mathrm{inr}\, p$，其中 p 的目标**就是** Φ_{i+1} 的数学状态。$B_{i+1} = B_i - 1 < B_i$，故读取该准确起点的回执 $(d', s', \ldots)$，返回 $(d' + 1, s', \ldots)$。目标偏移 $(i + 1) + d' = i + (d' + 1)$ 给出结算事件运输，$d' \le B_i - 1$ 给出 $d' + 1 \le B_i$。这定义了所有起点的 Type 回执，且每一步保留实际分支等式。

距离恰好为预算。先对 $h \le B_i$ 归纳。若 $h < B_i$，归纳得到 $B_{i+h} = B_i - h > 0$。结算要求当前表达式为常量，预算为零，所以实际动作必须是付款；其目标按实际帧的下一步给出 $B_{i+h+1} = B_i - h - 1$。于是

$$B_{i+h} = B_i - h \quad (0 \le h \le B_i), \qquad a_{i+h} = \mathrm{inr}\, p_h \quad (h < B_i).$$

回执的终点是实际结算，故 $B_{i+d} = 0$；回执已给出 $d \le B_i$，代入上式得到 $d = B_i$。特别地，零预算起点返回距离零，既不制造付款也不跳过结算。

完整状态与环境。每个 p_h 把原 Trace 接上一个真实执行步，保留原登记输入；归纳给出准确已付前缀及所有中间状态。可靠性给出终点常量值 $\mathrm{ev}_\rho e$。原环境读取器的登记相容律说明，在固定输入的数学推进处它读回同一环境；沿上述实际付款等式逐步应用该律，得到环境在 $0 \le h \le B_i$ 保持。没有使用该律来证明预算或距离。

原编译与出生。在终点实际等式 $a_{i+B_i} = \mathrm{inl}\, s$ 上展开原编译器：这次结算调用原责任准入，发射的 canonical next 是 active presentation of $\mathrm{nextBorn}(\Phi_{i+B_i}, \mathcal{P})$。密封运行的 `actual_node` 把它认回原运行的第 $i + B_i + 1$ 帧。故准入后继与原下一帧完整相等，不只是擦除或预算相等。

最后令 $u_0 = i$、$v_j = u_j + B_{u_j}$、$u_{j+1} = v_j + 1$。每个 v_j 带自己的上述 Type 回执，$v_{j+1} \ge v_j + 1$。归纳得 $v_j \ge i + j$，对任意原 tick t 取 $j \ge t$ 即有 $t \le v_j$；出生序列共尾。每个片段 $u_j, \ldots, v_j$ 的帧仍是原递归函数在这些索引的值。■

直接消费者与固定来源。总值出生环境使用回执中的实际结算（S5.2）；原包停点把每段完整付款前缀接入同一运行（S5.3）。`ACT/Completion.lean`、`Completion/Prefix.lean`、`Completion/Births.lean` 的 `receipt`、`receipt_distance_exact`、`receipt_next`、`receipt_compiles`、全部 prefix 与出生 cover 声明，来源 MV2；在 MV3 读取原构造与消费者。

S2 定理 8.12 与定理 7.7

S2.1 定理 8.12：实际动作生成下一问

对象。询问帧 Φ：询问状态、读取根编译器的完整原生程序、当前访问登记的原始输入 (ρ, e)、为每个原发生给出实际环境的读取器 E、因果深度 k。帧的数学状态是 $\Lambda_{\rho,e}$ 的已付状态 (e', t)；ρ' 为 E 在帧所在发生处的读数，$\varepsilon = \rho' - \rho$。

(1) 请求的值。残差请求 $e \ominus e' = e + \mathrm{linear}_{-\mathrm{id}}(e')$ 登记在 $\rho + \varepsilon = \rho'$ 上。按求值定义 $\mathrm{ev}_{\rho'}(e \ominus e') = \mathrm{ev}_{\rho'}(e) - \mathrm{ev}_{\rho'}(e')$。由可靠性 $\mathrm{ev}_{\rho}(e) = \mathrm{ev}_{\rho}(e')$，旧值为零。已付迹的关系像为 $e - e'$ (S0.3)，其在 ρ' 下的值由 (S0.2) 等于 $\mathrm{eff}_{\rho,\varepsilon}(e) - \mathrm{eff}_{\rho,\varepsilon}(e')$，与请求值一致。由 S0.4，这份证书在 $\mathrm{ev}_{\rho'}$ 下的典范残差为零，当且仅当它的值为零，即请求值为零。结算处 $e' = \mathrm{const}(\mathrm{ev}_{\rho} e)$，请求值是 $\mathrm{eff}_{\rho,\varepsilon}(e)$。

(2) 必然出生。$r(e \ominus e') = r(e) + \big(r(e') + 1\big) + 1$：加法节点贡献 1，线性节点贡献 1。此数为正，故初始状态不是常量，执行器的第一个动作是执行步；在出生编译中，这一步编译为责任准入（新责任的出生），而不是任何旧债行的付款。

(3) 递归帧与运行。定义帧的**秩** $\mathrm{rk}(\Phi) := (r(e),\, k+1)$，按字典序比较。下一帧由当前动作决定：

- 执行步：同一帧，深度加一，秩为 $(r(e), k+2)$；
- 结算：出生帧，旧状态取当前询问状态，程序取后继程序，登记输入取残差请求，深度归零，秩为 $(r(e) + r(e') + 2,\, 1)$。

两种情形都严格增大秩。帧序列 Φ_n（Φ_{n+1} 为 Φ_n 的下一帧）因此沿字典序严格递增。帧的擦除确定其秩：原始输入由根的观察读回（擦除保留词汇与访问，观察读回登记的原始输入），因果深度由擦除后访问的历史读回。若 Φ_m 与 Φ_n 擦除相同而 $m < n$，则秩相同，与严格递增矛盾。故擦除单射。逐支检查后继合法：执行步使用直接回答的后继律，结算使用责任准入的后继律，两者都保持同一根上生成的活法则。现成规范询问运行时在计数 n 的节点就是 Φ_n 的呈现：对 n 归纳，节点的激活后继等于下一帧。每个帧的查询纤维是单点（它唯一的查询就是该帧的登记输入），故不需要外部的未来请求表。

(4) 付款与出生分离。把实际节点限制到固定根的债过程：执行步分支在同一数学行上发射同债付款回执，其整账去向等于联合执行步的账演化（定理 6.10(2) 的读法）；预算为零处没有付款宏（定理 6.10(3)）；结算分支的出生是新的责任准入。■

形式对应。`JS/Native/ResidualRequest.lean`（`expression_eval`、`old_value`、`budget`、`action_is_paid`、`updated_value`、`residual_zero_iff`）；`JS/Native/Restructuring/Inquiry/Continuation/` 的 `Policy.lean`（`rank`、`Before`、`next_progress`、`frames_forward`）、`Inventory.lean`（`rank_eq_of_erasure`、`frames_erase_injective`）、`Occurrence.lean`（`successor_valid`）、`Runtime.lean`（`actual_node`、`macro_next`、`macro_next_preserves`）、`Payment.lean`（`actual_paid_receipt`、`no_paid_of_zero`）。提交 AB。

S2.2 定理 7.7：登记输入的共享运行与后继供料

对象。命题 7.6 的固定数据：旧询问状态（根、访问、审计协议、演算、查询类型、承担者与权威）、完备复内积空间 $\mathcal{H}$ 上安装的识别、该访问的一步及其处置 settle；另取根在该访问登记的原询问输入 ι：一个投影位置、它在该发生处的激活证明、分类等式与查询类型等式。

(1) 登记激活。按 settle 的输出分支。

全生成或联合残差支。命题 7.6 给出程序包与计算询问状态，后者的根是装入计算询问后的根。在该根上新增一条**查询投影律**：投影类型为单点；在发生 o' 处，激活当且仅当 o' 是本次访问的准确发生；载荷类型为计算查询；投影值是以 ι 的原查询为关联的计算查询。新根 r_ι 是原根加这一面投影；访问不变。登记输入 ι' 取这面投影、准确发生的激活证明与分类等式（分类在准确发生处按定义取激活支），查询类型等式按定义成立。询问状态 s_ι 的根为 r_ι，访问、审计协议与演算沿用旧状态；承担者经查询的关联读出；权威是旧权威沿新增投影面的提升；编译程序对每个查询取回答，回答面是程序包的结果面经投影嵌入。

于是 s_ι 对 ι' 的查询编译为回答（按定义）。回答面读出的表达式是程序包的整树程序，故解析器读回 T（定理 3.9(3) 与命题 7.6 的树等式）；值为 $\delta_{\tau(T)}$（定理 3.9(1)）；源状态是程序包的已付状态。旧状态对每个原查询 q 的编译面经同一嵌入读回，所读出的仍是旧编译的典范令牌。

其余四支。终账支返回根编译器在源发生处的整账编译，三个残差支返回残差对象本身，与命题 7.6 一致。

(2) 共享运行。以 s_ι 为询问状态、以“返回 ι' 查询的原始输入”的常值函数为读取器，构造数学询问运行时（定理 6.10 的所有者无关安装，加上 §8.5 的运行时）。所有者无关安装的原始输入就是该查询的原始输入；由定理 3.9(3)，其表达式读回 T。定理 6.10(3) 给出计费历史长度 $r(\mathrm{prog}_\kappa T)$，由定理 3.9(2) 改写为 $\beta(T)$；完成值 $\mathrm{ev}(\mathrm{prog}_\kappa T) = \delta_{\tau(T)}$。各计数处数学行的整账写回与原材料、各运行偏移处的实际查询、回答与后继，是数学询问运行时在相应计数与偏移的节点等式（定理 8.12(3)）。

(3) 后继供料。输入只取活根、访问 v、识别与审计协议。在非终账分支，步携带后继 succ 与等式 $\mathrm{next}(v) = \mathrm{succ.current}$；令 $v^+ := v.\mathrm{next}(\mathrm{succ})$。识别在 v^+ 生成一步并经 settle 分类，(1)(2) 的构造在 v^+ 重新给出供给与运行。终账支只返回整账编译。每个非终账支把前拍的程序包（对齐两支）或残差（三个残差支）与下一运行配对返回。

两拍的整账写回与下一当前：步的定义包含 $\mathrm{whole}(\mathrm{step}(v)) = \mathrm{generatedLedgerAt}(v.\mathrm{current})$ 与 $\mathrm{nextCurrent}(\mathrm{step}(v)) = \mathrm{generatedNextCurrentAt}(v)$，在 v^+ 同样成立。后继的目标发生等于根在 v^+ 当前的规范发生（目标发生是编译器为下一当前发射的事件），历史是发生的函数，故两处历史相等。■

直接使用。§9 的联合程序（定理 9.2）以同一识别与同一后继为输入；定理 9.8 的默认运行在每个访问消费 (3) 的下一运行。

形式对应。`DJ/Input/{Source,Consumer}.lean`、`DJ/Input/Runtime/{Source,Consumer}.lean`、`DJ/Next/{Source,Consumer}.lean`；低源询问 `JS/OwnerFree/Installation/Math/{Inquiry,Frame}/Source.lean`。提交 AC。

S2.3 定理 8.14：同次差词、独立收费与源环境方块

两份实际边界。取原源族生成的下一包，而后取其实际 grandnext 发生；b_Ψ 是原终端执行的配对提升，b_μ 是原下一编译执行的完整边界。两份 Trace 各自给出原下一历史关系闭包中的成员证明。它们没有按读数选择发生。原 after scope 中，记原请求点为 r，原终端值为 Ψ，原写入值为 $\mu = (\Psi, \sigma)$，常值嵌入为 c。原编译的两个仿射边界公式是

$$Q^+(b_\Psi) = r - c(\Psi, 0), \qquad Q^+(b_\mu) = r - c(\mu).$$

因此 $d = b_\Psi - b_\mu$ 在同一关系闭包中，且线性与加群消去给出

$$Q^+(d) = (r - c(\Psi, 0)) - (r - c(\mu)) = c(\mu - (\Psi, 0)) = c(0, \sigma).$$

after scope 的实际环境读取这个常值，于是独立差词查询的求值为 $(0, \sigma)$。这里的历史闭包保存两次原执行的责任来源；更新后读数由 after scope 读取，没有假设更新前的核继续是更新后的核。

系数程序自己的费用。把 d 的有限支撑按其原支撑列表写成 $(e_1, a_1), \ldots, (e_k, a_k)$。定义 $N_0(e) = \mathrm{const}(0)$、$N_{n+1}(e) = e + N_n(e)$；对整数 a，正系数展开为 $N_a(e)$，负系数 $a = -(n+1)$ 展开为 $-N_{n+1}(e)$（一枚原负号线性节点）。由 n 归纳，

$$\mathrm{ev}_\rho N_n(e) = n\,\mathrm{ev}_\rho e, \qquad r(N_n(e)) = n(r(e) + 1).$$

故整数展开 $E_a(e)$ 满足

$$\mathrm{ev}_\rho E_a(e) = a\,\mathrm{ev}_\rho e, \qquad r(E_a(e)) = |a|(r(e) + 1) + \mathbf{1}_{a<0}.$$

令 $F([]) = \mathrm{const}(0)$、$F((e,a) :: L) = E_a(e) + F(L)$。列表归纳给出

$$\mathrm{coeff}(d) = F(\mathrm{supportList}(d)), \qquad c(d) = \sum_{(e,a)\in \mathrm{supportList}(d)} (|a|(r(e)+1) + \mathbf{1}_{a<0} + 1),$$

以及 $\mathrm{ev}_\rho(\mathrm{coeff}(d)) = \mathrm{ev}_\rho d$、$r(\mathrm{coeff}(d)) = c(d)$。空差词费用为零；结算后的 residual 出生仍由其自己的正预算保证。定理 6.10 执行这个原系数程序，给出长度 $c(d)$ 的实际 Trace、$c(d)+1$ 个状态及自己的结果 $(0,\sigma)$，不消费另一计算的收费证书。

安装与 actual next。将该 Raw、结果、Trace 和全部偏移材料登记于原 root 的投影。实际 query incidence 认回该登记；它的原 answered 编译保留查询结果，然后残差出生编译在同一 root、同一 visit 生成责任准入。准入投影返回刚登记的完整材料，旧投影按完整编译的余项继承。故其 effect、inverse residual、charge、Trace、root、whole ledger 与 literal next 来自同一编译路径。准确后继在原 $k+2$ 包上供给下一 Raw，不从预设包表取值。

反馈与完整联合库存。设 q 为该源族的实际完整查询，q' 为它的已付端点，$h = q \ominus q'$，β 为其 Trace 边界，θ 为原 action binding。可靠性给出 $\mathrm{ev}_\rho h = 0$；代换求值与更新等式给出

$$\mathrm{ev}_\rho(h[\theta]) = \mathrm{eff}_{\rho,\,\rho\circ\theta-\rho}(\beta), \qquad \mathrm{ev}_\rho(h + h[\theta]) = \mathrm{eff}_{\rho,\,\rho\circ\theta-\rho}(\beta).$$

这里 $\rho\circ\theta$ 表示在 ρ 中求值原绑定得到的环境。$r(h) \ge 2$，$r(h+h[\theta]) \ge 3$，故这份完整反馈实际付款；其完成 Trace 长度为自身剩余计数。S0.3 给出同一值为 β 的 updated canonical residual 读数。有序混合效应按 S0.7 的逐步伸缩等式给出同一完整结果。库存定义为

$$\mathrm{seed}(H_{\mathrm{old}}, \mathrm{seed}(\mathrm{seed}(H_{\mathrm{dispatch}}, \mathrm{exposure}(T_q)), \mathrm{exposure}(T_{h+h[\theta]}))).$$

左右种子嵌入分别保存旧库存、原查询每个 paid event 和反馈每个新 paid event；原 stock-to-joint 接口把这些成员证明交给实际 jointStock。外层 reader、内层 query 和 whole-next 正是从该库存读取的原消费者。

有限 Incoming 与环境方块。原真实配置的计算读取器取 $\mathrm{Incoming}(\Phi, D)$，其中 D 是原查询 datum。若 pairInventory 为空，返回 $D.\mathrm{reader}(o)$；若非空，选该库存的原 expression 或关系词系数表达式 g，返回

$$(D.\mathrm{reader}(o).\mathrm{environment},\ D.\mathrm{reader}(o).\mathrm{expression} + g).$$

两支的环境都由定义等于原查询环境。默认 actionReader 用 `calculationReader.getD(queryReader)`：配置为空时两边相同，配置为上述 Incoming 时由这两支相同。因此真实源族在每个发生上生成通用 `sourceActionEnvironment` 合同，而不把它留作新输入。Incoming 不调用旧 calculationReader，故新 g 只加一次。配置只替换 calculationReader，原 queryReader、环境解码器、两种库存回调与 receiver/seed 均保留。

实际 residual 接收器的预算是 $B = r(e) + r(e') + 2 \ge 2$；其首个执行步后预算为 $B - 1 \ge 1$。故接收器实际 paid，证明来自 source residual 和首步严格下降，而非 query charge 与 calculation charge 的同名字段。上述 source square 再把 actualNativeEnvironment 与原 query environment 对齐，完成原写入／Born ／下一查询的环境链。■

固定来源。差词与安装来源 PR；joint carrier 回写来源 MF，采用 `Cx/Profile/Finite/Reader/Difference/{Source,Inquiry/Source,Inquiry/Installation,Inquiry/Input,Inquiry/Receipt,Inquiry/Compiler,Inquiry/Actual,Actual}.lean`、`Profile/Producer{,/Consumer}.lean`、`Mixed/{Effect,Receipt,Consumer}.lean`、`Difference/Actual.lean` 与 `Psi/TerminalTransport.lean`。源配置的公开修复在 H0 的 `EffectInstalled.lean`、`GeneratedEnvironment.lean`、`FiniteCalculationInstallation.le`

an、FiniteIncoming.lean、ContextualFactory.lean、ActualNativeEnvironment.lean、Environment Erasure.lean，固定读取 H0-S；原历史对象与公开修复不互换证明来源。

S3 定理 9.2 与定理 9.3

S3.1 定理 9.2：同一程序承载完整时间、共同历史与作用差

对象。活根、时间访问 v、识别、审计协议；$\mathrm{step}(v)$ 落在全生成或联合残差支，带后继 v^+、历史转移与配对对齐；T 为命题 7.6 的对齐构造子树。

(1) 时间码。根的时间访问由三种归纳历史之一给出：自初始当前有限步可达，封闭共尾，或共尾之后可达。对每一种，归纳类型 Hist、Post 与原可达性证据之间有互逆的编码与解码，按构造子逐一定义并逐一验证。时间码 $c = (\text{当前}, \text{历史码})$；enc、dec 逐分量取上述编码与解码，故互逆，enc 单射。$\mathrm{act}(c) = (\mathrm{rcpt}(c), \mathrm{nx}(c))$ 的第一分量包含 c 本身，故 act 单射。后继：时间访问的后继由编译器的下一当前与历史延长给出，编码在延长上逐构造子交换，于是 $\mathrm{enc}(v^+) = \mathrm{nx}(\mathrm{enc}\, v)$。

(2) 联合程序。节点类型取（T 的对齐节点，时间码），T_J 为 T 的每个节点配上 $\mathrm{enc}(v)$。结果类型取五元组

$$(\text{效应折叠值，收据树，两段历史的原始材料，两侧评价器输入，两侧暴露计划}).$$

构造子 $\kappa_J(n, \vec{y})$：第一分量是原依赖效应的单节点折叠 $\mathrm{effectFold}(n_1, [\, y_{i,1}\,])$；第二分量是以 $\mathrm{act}(n_2)$ 为根、以各子结果的收据树为子树的树；后三分量分别取两段历史的原始材料、两侧评价器输入与两侧暴露，与子结果无关。程序 $P = \mathrm{prog}_{\kappa_J} T_J$（定义 3.8）。环境 δ 把每个节点变量送到其状态点；δ' 把每个节点 n 送到 δ_{n^+}，$n^+ := (n_1, \mathrm{nx}(n_2))$（无后继时保持 n_2）。

值。对 T_J 与子列作互递归：

$$\mathrm{ev}_{\delta'}(P) = \delta_{\mathrm{fold}_{\kappa_J}(T_J^+)},$$

其中 T_J^+ 是把每个节点换成 n^+ 的树——这与定理 3.9(1) 的证明相同，只是基点换成 n^+。由 S0.2，配对环境 $(\delta, \delta' - \delta)$ 下 $\widetilde{P}$ 的值为 $(\mathrm{ev}_\delta P,\ \mathrm{eff}_{\delta, \delta'-\delta} P)$；更新等式（命题 3.3）给出 $\mathrm{eff}_{\delta,\delta'-\delta}(P) = \mathrm{ev}_{\delta'}(P) - \mathrm{ev}_\delta(P)$。合起来，值为 $(\delta_F, \delta_{F'} - \delta_F)$，$F = \mathrm{fold}_{\kappa_J} T_J$，$F' = \mathrm{fold}_{\kappa_J} T_J^+$。

分量。对 T_J 归纳：F 的第一分量只依赖各节点的第一分量与子结果的第一分量，恰为 $\mathrm{fold}_{\mathrm{effectFold}}(T) = \tau(T)$；$F$ 的第二分量是 T_J 逐节点取 act 的像树，其根为 $\mathrm{act}(\mathrm{enc}\, v)$，由 (1) 解码回 v。

计费与付款。由 S0.2 与定理 3.9(2)，$r(\widetilde{P}) = r(P) = \beta(T_J)$，而 T_J 与 T 形状相同，故 $\beta(T_J) = \beta(T)$。把定理 6.10 施加于读取器 $o \mapsto (\text{配对环境}, \widetilde{P})$：已付迹长度为 $\beta(T)$，每个计数 $k < \beta(T)$ 有严格付款，目标预算不回充，续行良基。

逆读与后继。在每个计数，安装面保留的表达式是 P，解析器读回 T_J（定理 3.9(3)），去掉码分量即读回 T。读回的根码解码为 v，其生成下一当前就是根在 v 处的生成下一当前。更新态射 $(\mathrm{id}, \mathrm{ev}_{\delta'})$ 从 ev_δ 指向 $\mathrm{ev}_{\delta'}$；源侧取旧迹的关系词，目标纤维元素取旧、新两条迹关系词之和。由引理 S0.5，提升残差为零当且仅当目标纤维元素是旧纤维中某一元素的像；这份残差随结果原样保留。

(3) 共同历史。源、目标历史都是同一历史法则在两个发生处的历史；共同历史的事件、支撑、生成元闭包与关系闭包取两侧之并。两条转移把各侧的生成元按定义嵌入。完成映射把一侧生成元闭包中的词 u 送到共同完成中同一个词的类，因此像为零当且仅当 u 属于共同历史的关系闭包。两侧评价器取历史法则在两个发生处的忠实评价，在每个生成元上读回原忠实面的值；关系可靠性两侧等价。完成载体 Src、Tgt 之积带有由两条转移诱导的作用与观察，构成作用模型；配对与对偶作用由历史的标量配对给出；测量 $\mathrm{Src} \times \mathrm{Tgt} \to \ell^2(\mathcal{H} \times \mathcal{H})$ 与该空间上的线性等距演化组成积分—相干联合作用数据。

(4) 作用差的逆判据。记 ∂_s, ∂_t 为两侧的微分（测量），转移满足 $\partial_t \circ \iota = \partial_s$。

若存在 r 使 $\partial_s r = \partial_s a_s$ 且 $\iota(r) = a_t$：则 $d = \iota(r) - \iota(a_s) = \iota(r - a_s)$ 属于 $\operatorname{range}\iota$，像残差为零；$\partial_t d = \partial_s(r - a_s) = 0$，观察残差为零。

反之，像残差为零给出 r_0 使 $\iota(r_0) = a_t$，于是 $d = \iota(r_0 - a_s)$，观察残差 $\partial_t d = \partial_s(r_0 - a_s) = 0$，故 $r_0 \in \mathrm{Fib}_{\partial_s}(a_s)$。

计算程序是三输出表达式：差 d，它的测量，及它在余核中的类；其剩余计数为 25。定理 6.10 给出 25 步已付迹，每步严格付款，续行良基，整账与下一当前在每个计数保持。

(5) 字面后继。$\mathrm{step}(v)$ 的目标发生是编译器为下一当前发射的事件，等于 v^+ 当前的规范发生，而后者就是 $\mathrm{step}(v^+)$ 的源发生。历史、生成元闭包、关系闭包、暴露与暴露树都是（当前，发生）的函数，故两两相等；目标词沿生成元闭包的相等运到下一源词，底层的词不变。■

形式对应。`Occurrence/Temporal/{Source,Action/Source,Action/Inverse}.lean`；`Occurrence/Temporal/History/Common/{RootSource,Closure,Evaluator/Source,Action/Source}.lean`；`DJ/Joint/{Source,Laws,Consumer,TargetDifference,TargetProgramme,TargetConsumer}.lean`；`DJ/Next/{Material,Pursuit}.lean`。提交 AD。

S3.2 定理 9.3：全部作用者的共同恢复

对象。定理 9.2 的数据与一个计数。源、目标配对展开树分别以全部配对位置为节点，每个节点带暴露 (M_p, A_p, U_p)。作用者是树迹中的成员；节点 Hilbert 的指标取树迹的位置（同一载荷重复出现时分别计入）。

(1) 全词模型。联合载体取两侧载体之积；字母 ℓ 的作用是该作用者的载体作用（源作用者作用于第一分量，目标作用者作用于第二分量，“同时”同时作用两侧的根），根作用者的作用就是原步的源、目标暴露作用。读出 rd 是粗读口与全部作用者测量的积。引理 S0.6 给出商 Mod、纤维刻画与完成等价；完成载体中两元素相等当且仅当它们对所有后续词的读出相同，这是 S0.6(ii) 在完成一侧的表述。

(2) 有序效应历史。取 $W = \mathcal{H}^{\text{作用者}}$，$U_\ell$ 对被选作用者的分量施加 U_p、其余分量不变（“同时”对两侧根分量施加），M 为各作用者的测量。引理 S0.7 给出总残差等于各步残差经其后演化运送之和，历史长度为 $|w|$；单字母情形，总残差就是该作用者的协变效应 $U_p M_p(x) - M_p(A_p x)$。

(3) 节点 Hilbert。测量 $\mathrm{meas} : \mathrm{Mod} \to \ell^2(\text{位置}; \mathcal{H})$ 逐位置取该作用者的特征（作用者测量经模型读出）。在 ℓ^2 中范数平方是各坐标范数平方之和，故

$$\|\mathrm{meas}(x)\|^2 = \sum_p \|M_p(x)\|^2.$$

字母在 ℓ^2 上的演化逐坐标取 U_p 或恒等，故是线性等距。节点效应按协变耦合残差的定义逐坐标展开。

(4) 未来恢复与三面。

未来恢复。设 $x \in \mathrm{Mod}$ 被粗读口与全部未来读口同时送到零。全部未来 $\mathcal{H}$ 读口为零意味着：对每个前缀界与每个词，每个作用者在词作用后的测量与零点处相同。粗读口为零给出粗读出在每个词作用后与零点相同。于是 x 与 0 在每个词作用后的全部读出相同，由 (1) 得 $x = 0$。合成映射单射，Mod 与其像线性同构，两个方向的复合是恒等。

三面。以 $I = \mathrm{Mod}$（作用为模型推进）、$H =$ 全部前缀界上的 ℓ^2 族（作用为逐界演化）、$Q =$ 粗模型（作用为粗推进），c 为未来读口，m 为粗限制，作引理 S0.8 的完整轨道载体 $\mathcal{C} \subseteq I \times H \times Q$。积分面与粗读面就是第一、第三分量。相干面把第二分量（全部前缀界上的族）送到沿前缀界限制的逆极限：逐界限制与演化交换，故这是相容族的完成映射，并与每个字母的演化交换；完成映射的每一界坐标就是第二分量在该界的值，所以相干面为零当

且仅当第二分量为零。于是三面同时为零的元素三个分量都为零，即为零：三面联合单射，在像上双逆，并与词作用交换（S0.8(ii)）。

(5) 有偿恢复。对点、种子词、历史词与查询词 q，所有者无关计算执行的是按查询词生成的程序，其剩余计数由程序预算引理给出为 $|q|+3$；定理 6.10 给出已付迹长度 $|q|+3$，计算值等于下一值（值读回引理）。恢复映射施加于作用后的元素，第一分量按 (4) 的同构读回下一值。配对库存作为新生库存的一部分进入下一次出生，同债不回充（定理 6.10(2)）。■

形式对应。`AW/{Source,Complete/Source,EffectHistory/Equation,EffectHistory/Source}.lean`；`AW/NodeHilbert/{Source,Future/Recovery/Source}.lean`；`AW/NodeHilbert/Future/Wave/{Words/Kernel,Full/Source,Recovery/Source,Recovery/Consumer,Incidence/Consumer}.lean`。提交 AD。

S3.3 定理 9.11：同一实际材料的六面

共同材料。取原实际帧的 Paid material M，其中保存发生 o、环境 ρ、原 Raw e、已付状态 e'、完整 Trace $T : e \to e'$、owner 与原增量 ε。关系词 $\eta = T.\mathrm{relationWords}$ 是原 Trace 逐步生成的词，不从最终读数重建。每步关系像为 $\delta_{e_h} - \delta_{e_{h+1}}$，相加给出

$$\mathrm{rel}_\rho\,\eta = \sum_{h<|T|} (\delta_{e_h} - \delta_{e_{h+1}}) = \delta_e - \delta_{e'} =: b.$$

令 H 为该实际 paid 前缀的 prefixHistory；每个关系证据由此前缀产生，故其和属于 H 的关系闭包。因此实际关系 $\sigma = (b,\, b \in H.\mathrm{relationClosure})$ 是这次历史中的具体元素；将 b 投影到实际历史完成载体得到零。

六面与复形。代数面保留 $(e, e', e \ominus e')$ 及完整系数配对；组合面保留 T、每个步骤的 exposure 和位置；拓扑面是上述实际历史与其闭包；逻辑面是下一复形的链条件。令第一微分 d_1 为关系闭包的包含，第二微分 d_0 为历史完成投影：

$$C_1 \xrightarrow{d_1} C_0 \xrightarrow{d_0} \widehat{C}_0, \qquad d_1(\sigma) = b, \qquad d_0 d_1(\sigma) = 0.$$

在实际历史关系闭包上同样成立，所以微分复合为零。关系面保存这份实际 σ，cochain 面保存这份实际复形与同一边界。六面用同一个 M 构造，其发生与 Raw 读出按定义相同；不从四个各自存在的面拼出一个共同发生。

效应与完整词恢复。S0.3 给出

$$\mathrm{ev}_{\rho+\varepsilon} b = \mathrm{eff}_{\rho,\varepsilon} b = \mathrm{ev}_{\rho+\varepsilon}(e \ominus e').$$

以更新求值的核作商，典范类为零恰等价于右边为零；类到像的标准同构把它读回原 native value，给出 inverse/residual 的准确判据。完整词的恢复使用原完整词读口，核为零：若所有基词系数的读出为零，各系数为零，故词为零。余像恢复因此在任意 w 上满足 $\mathrm{rec}(\mathrm{can}(w)) = w$，组合面仍保留完整 Trace。

越过 literal Born。原准入的库存回调以 preserve 将这份 paid 材料嵌入实际 Born，且 stock outcome 的 Trace 是同一原 T。原结算编译给出下一 root/current 的完整身份；沿这条身份运输 installed Raw 的投影、active、classifier、payload 四项等式，得到下一实际查询的 Raw，source material 的 raw/environment/state/Trace/owner 同时运输。若 Born 的实际环境是 ρ_B，注册请求仍取原登记环境 ρ，其 increment 为 $\rho_B - \rho$。这一运输仅改变 increment：原 σ, b, T 均保留；更新效应为 $\mathrm{eff}_{\rho,\rho_B-\rho} b$。在 ρ_B 下新请求规范化后生成新 σ_B 与新边界；原库存保留二者的来源，未断言 $\sigma = \sigma_B$。■

固定来源与消费者。MV3 的 `ACT/Receipt/Configured/Writeback/Successor/FourFace/{Source,Consumer,Native/Source,Native/Consumer}.lean`；`Operations/Inquiry/Cofinal/Native/{Source,Co`

nsumer,Registered/Source,Next/Source,Next/Consumer,Installed/Source,Installed/Consumer,Cofinal/Source,Cofinal/Consumer}.lean。原下一包 installed 材料由 S5.4 的整词作用与全部未来模型消费。

S4 定理 9.5–9.7

S4.1 定理 9.5：母准入

对象。询问帧 Φ 与源程序 $\mathcal{S} = (L, \text{读取器}, \dots)$（定义 9.4）。共享运行时把 Φ 与 $\mathcal{S}$ 的读取器装成询问状态并回答其唯一查询；s 是这一已回答状态，o 是其当前发生，ρ' 是共享运行时字面下一状态的查询原始输入的环境。

(1) 材料与值。材料按定义装配：环境 ρ 与表达式 e 取读取器在 o 的输出，增量 $\varepsilon = \rho' - \rho$，已付状态 (e', t) 取共享运行时在 o 的结果，承担者取 s 对该查询的承担者。登记输入是定义 8.11 的残差请求，环境为 $\rho + (\rho' - \rho) = \rho'$。值与残差由 S2.1(1) 给出。

(2) 必然准入。残差请求的初始状态若为结算，其预算为零（执行债律在结算处预算为零）；但残差请求预算为 $r(e) + r(e') + 2 > 0$（S2.1(2)），矛盾。故源的第一个动作是执行步。出生程序由旧状态、查询、登记输入、源包、权威与这一步生成；在 s 的根与访问上，编译程序把唯一查询送到该出生程序在精确时间因果事件上的生成，即责任准入，等式按定义成立。

(3) 目标。准入目标携带：字面下一当前等于目标状态的（词汇，根，访问）；整账的第一次写入；每个旧投影在目标上的结果与原结果相等（异构相等）。后继合法性由“主动节点在准入后继下保持生成活法则”的一般准则给出，其前提正是上述字面下一当前与编译等式。

(4) 新生帧。新生帧取 s 为旧状态、残差请求为登记输入、读取器环境、深度零，库存为已付迹的全部暴露。其当前状态按定义就是准入目标状态。定理 8.12 的运行时以它为初始帧：初始节点的擦除等于目标呈现的擦除；每一拍的宏后继与活法则保持是 S2.1(3) 的节点等式；不回充与良基是 S2.1(4) 与定理 6.10(2)。

(5) 下一内容。由 (S0.1) 与更新等式，已付结果的（旧，效应）之和等于程序在更新环境中的值：

$$\mathrm{ev}_{\rho}(e) + \mathrm{eff}_{\rho,\varepsilon}(e) = \mathrm{ev}_{\rho+\varepsilon}(e).$$

新生帧在每个发生处的环境由这一值的环境读出给出。宏历史程序满足 $\mathrm{next}(\mathrm{hist}_k) = \mathrm{hist}_{k+1}$；初始帧的活跃环境已是第 k 拍历史的下一值，即第 $k+1$ 拍，新生帧再推进一拍，故为第 $k+2$ 拍。新生库存取旧库存与已付暴露的并列种子，两者都是它的部分（左、右嵌入）。

(6) 原生移位。同一阶段的查询帧经原生游标移位得到新的帧；对它施加 (1)–(4)，即得编译、整账首写、后继合法、字面后继、每个偏移的运行、不回充与良基；移位后环境就是游标移位的实际环境，效应与逆读由残差请求读出。■

形式对应。ACT/Programme.lean、ACT/Receipt/{Source,Execution,Compiler,Consumer,Runtime}.lean、ACT/Target.lean、ACT/Receipt/Faces/{Source,Action,Consumer,Native/Source,Native/Consumer}.lean；下一内容 AW/NodeHilbert/Future/Wave/Incidence/Environment/{Source,Consumer}.lean。提交 AD。

S4.2 定理 9.6：有类型库存越过准入

对象。定理 9.5 的设置；物理原始输入 (ρ_P, e_P)（变量取物理类型 Var_P）与已付结果，低层原始输入 (ρ_L, e_L)（变量取 L）与结果。绑定 $\sigma : \mathrm{Var}_P \to \mathrm{Expr}_L$，$\sigma(x) = \mathrm{const}(\rho_P(x))$。

(1) 绑定保值。对表达式结构归纳证明 $\mathrm{ev}_{\rho_L}(e[\sigma]) = \mathrm{ev}_{\rho_P}(e)$：变量处 $\mathrm{ev}_{\rho_L}(\mathrm{const}\,\rho_P(x)) = \rho_P(x)$；常量处两边相同；加法、线性、双线性节点由归纳假设逐项相等。形式词按线性延拓。低层查询 $(\rho_L, e_P[\sigma])$ 的值因

此等于物理查询的值。代换迹是物理已付迹沿 σ 的代换，起点为 $e_P[\sigma]$，关系边界随之代换；物理已付状态是常量，代换后仍是常量。任何以常量为终点的执行迹，其长度等于起点的剩余计数（命题 3.7(1)），故代换迹的长度为 $r(e_P[\sigma])$。

(2) 库存登记与读回。完整材料（两侧原始输入与结果、绑定、两份已写库存与代换迹）作为一面投影安装在帧的基础根上，载荷类型就是材料的类型。母准入目标的投影律继承旧投影律（定理 9.5(3)），在这面投影处的结果与原结果异构相等；类型相等由同一异构等式给出，沿它把结果转换为材料类型，即得到材料本身。

(3) 新生库存。物理库存中的每个事件经 σ 代换迁入低层语言，迁入后与低层库存并列成为新生库存；左、右嵌入保证两侧每个事件都在其中。新生帧按定理 9.5 运行；下一库存是新生库存的扩展，坐标映射在 mathNext 与 nextBorn 两种动作下都单射，且逐事件读回原事件。

(4) 逆读。以新生库存的事件为指标，词族 W 的联合映射 $W \mapsto (\mathrm{index} \mapsto (\mathrm{ev}_\rho, \mathrm{eff}_{\rho,\varepsilon})(W_{\mathrm{index}}))$；实际词族的典范残差为零当且仅当其像为零（S0.4），而像就是结果值。下一拍的逆读是更新态射诱导的映射（S0.4）。共尾的每一拍：关系词在该拍环境下的配对值，读出下一拍场在偏移零处的值；已付迹长度等于原始输入的剩余计数。■

形式对应。`ACT/Receipt/Inventory/{Source,Laws,Installation}.lean`；`Inventory/Born/{Source,Consumer}.lean`；`Inventory/Born/Vector/Fibre/{Source,Consumer}.lean`；`Inventory/Born/Coordinates/Actual/{Source,Query}.lean`；`Coordinates/Actual/Cofinal/{Source,Consumer}.lean`。提交 AE。

S4.3 定理 9.7：词—状态配对与有偿前缀

对象。运行时及其源；状态 s 的旧环境 ρ_s 与增量 $\varepsilon_s = \rho_{\sigma(s)} - \rho_s$。轨道变量的绑定 σ 满足两条读回：$\mathrm{ev}_{\rho_s}(\sigma x) = \rho_{\sigma(s)}(x)$ 与 $\mathrm{eff}_{\rho_s,\varepsilon_s}(\sigma x) = \varepsilon_{\sigma(s)}(x)$。

(1) 配对。$\langle w, \cdot\rangle$ 在基点 δ_s 上取阶段库存 $(\mathrm{ev}_{\rho_s} w, \mathrm{eff}_{\rho_s,\varepsilon_s} w)$，由自由模泛性质（命题 2.3）唯一线性延拓；对 w 线性由求值与效应的线性给出。

(2) 协变与像作用。在基点上：

$$\langle w[\sigma], \delta_s\rangle = (\mathrm{ev}_{\rho_s}(w[\sigma]),\ \mathrm{eff}_{\rho_s,\varepsilon_s}(w[\sigma])) = (\mathrm{ev}_{\rho_{\sigma(s)}} w,\ \mathrm{eff}_{\rho_{\sigma(s)},\varepsilon_{\sigma(s)}} w) = \langle w, A\delta_s\rangle,$$

中间一步是代换引理：代换后求值等于在“变量取代换式之值”的环境中求值，效应同理；两条读回把这个环境认作 $\rho_{\sigma(s)}$ 与 $\varepsilon_{\sigma(s)}$。两边对 $\mathbb{Z}[S]$ 线性，故整个载体上 $\langle \sigma^*(\cdot), \cdot\rangle = \langle \cdot, A(\cdot)\rangle$。这说明 (σ^*, A^*) 是从配对到自身的态射，由 S0.4 诱导余像上的作用；读出公式在 $\mathrm{can}[e]$ 上由定义给出。

(3) 迭代即时间。对 n 归纳：$n = 0$ 平凡；归纳步用 (2) 的基点等式于第 d 拍，再对 $d+1$ 用归纳假设。$\mathrm{act}^n[e] = [e[\sigma]^n]$ 由诱导映射与典范映射交换得到。

(4) 前缀计费。第 i 个前缀执行 $\widetilde{e[\sigma]^i}$，由 S0.1 与 S0.2，迹长 $r(e[\sigma]^i)$；求和得 Σ_n。批量查询把 $n+1$ 项装入向量值语言：每一项经一个坐标嵌入节点与一个累加节点进入，剩余计数为各项之和加 $2(n+1)$；执行长度等于剩余计数（命题 3.7(2)）。定理 6.10 给出逐步严格付款与同债谱系。

(5) 调用恢复。调用帧在计数 $c+i$ 处的实际查询材料给出该处环境；所需快照的旧值与增量逐项取自这些环境（增量为相邻两处之差）。由此组装的批量环境与轨道批量环境按定义逐坐标相等，故两个查询的结果值相等；每个调用的已付迹长度等于其剩余计数（定理 6.10(3)），求和得调用计费等于调用成本。■

形式对应。`Operations/Inquiry/Context/Native/Pairing/{Source,Orbit/Source,Orbit/Action,Orbit/Iteration,Orbit/Iteration/Batch,Orbit/Iteration/Current}.lean`；调用与历史 `…/Batch/Calls/{Source,Consumer,History/Source,History/Consumer}.lean`（完整前缀见正文附录 D.2）。提交 AE。

S4.4 定理 9.9：完整恢复、两库存与 Trace 位置

原模型先恢复整个载体。记原密封运行状态集为 S，$\mathcal{F} = \mathbb{Z}[S]$。原完整作用词模型的源映射为 q；它的 completeRead 包含空作用词上的原整词读取，故 $\mathrm{rec}(q(w)) = w$ 对每个 $w \in \mathcal{F}$ 成立。定义 O 在基状态上读取其实际登记帧，线性延拓为 $O(\delta_s) = \delta_{\mathrm{readFrame}(s)}$。密封运行的状态在第 i 拍储存原递归索引，节点等式与擦除单射给出 $\mathrm{readFrame}(s_i) = \Phi_i$。原完整模型作用的点等式 $Aq(\delta_{s_i}) = q(\delta_{s_{i+1}})$ 因而给出

$$O\mathrm{rec}(q(\delta_{s_i})) = \delta_{\Phi_i}, \qquad O\mathrm{rec}(Aq(\delta_{s_i})) = \delta_{\Phi_{i+1}}.$$

对帧的 $(\mathrm{inventory}, \mathrm{pairInventory})$ 作 mapDomain，得到准确两库存的单点词。若第 i, j 个模型点相等，先恢复得到相同原状态，再读节点和擦除得到 $i = j$；这里的单射由原严格帧秩给出，既往已付状态没有在 Frame 限制前丢失。

完整 stock。原查询的原始输入、value、Trace、effect、inverse、decoder、whole 与 literal next 均由相同安装读取器在实际发生处给出。计算读取器为空时它就是 registered Raw；有计算读取器时使用那个实际读取器，不能把两者无条件等同。原 stock 程序只把这些材料交给母准入，并保留原两库存回调。Type 完成回执指定的结算取出它实际的 Born，原密封节点在距离加一处恰读回这份完整帧。

逐位置嵌入。设 H_Q 为本次查询的 source written exposure, H_o 为 old written, H_m 为迁入的 written, H_P 为 physical written。原完整 written 与 Born 库存是

$$H_C = \mathrm{seed}(\mathrm{seed}(H_o, H_Q), H_m), \qquad H_B = \mathrm{seed}(H_P, H_C).$$

原 source exposure 与 actual query exposure 的完整树等式先把 $j \in \mathrm{Fin}(|H_Q|)$ 运到 sourceWritten。随后三个并列嵌入为：H_Q 进内 seed 的右侧、内 seed 进 H_C 的左侧、H_C 进 H_B 的右侧。原 seed 的左右读回等式逐层复合，故

$$H_B.\mathrm{trace}[\iota(j)] = H_Q.\mathrm{trace}[j], \qquad \iota(j) = j + |H_o| + |H_P| + 3.$$

每个 seed 的接缝各占一位置，故偏移为三。若 $\iota(i) = \iota(j)$，在自然数中取消共同偏移得 $i = j$；若 $i < j$，加同一偏移得 $\iota(i) < \iota(j)$。因此该位置映射单射且严格保序，两个同值事件也不合并。

原后继的两个读取位置。实际结算的 debtAdmission 恢复准入初访上的完整材料，也给出完整 next 包。旧初访的 after-initial read 保留原安装材料；新 current 的 after-current read 是原编译送出的下一查询。各自沿对应的 full-current 身份运输，读回各自原 reader。没有用“同一份源材料”把两个不同访问位置视为同一个查询。■

固定来源。MV2 的 `Operations/Inquiry/Context/Native/Frame/{Source,Stock,Stock/Return,Stock/Coordinates,Stock/Birth}.lean`，在 MV3 读取全词、model-next、inventory-next、source-exposure、intoBorn、after-initial 与 after-current 的原声明及直接消费者。

S5 实际总值、完整续跑与原整词作用

S5.1 定理 9.8：默认运行执行源自产请求与子树

(1) 源自产请求。取定理 9.2 的联合程序根与已安装读取器，作所有者无关安装的源询问帧，种子取登记表达式生成元的零展开。默认运行时的源程序以“本发生的原始请求”为读取器：在发生 o 处，读取器返回为 o 生成的请求表达式与环境。把定理 9.5 施加于这一源程序：查询原始输入就是安装面读出的请求（按定义）；结果值等于请求的执行值（定理 6.10 的完成值，连同 ev 的可靠性）；已付迹长度等于登记输入的剩余计数（定理 6.10(3)）；新生配

对库存按定理 9.5(5) 由既往已写与本次已付两部分组成；每个计数的实际查询、回答与后继是运行时的节点等式。账根相同是构造上的恒等。分支版本的核心表达式与关系表达式各自成为二元批量的一个坐标，批量执行的迹与代换迹进入初始库存。

(2) 子树进入主表达式。主表达式是三项之和：原查询表达式、旧码处子树、新码处子树，各项经坐标嵌入进入输出类型。某码处的子树：若识别在该码的访问没有后继，子树为零常量；否则为该码处子程序原始输入的嵌入。于是

$$\mathrm{ev}(\text{主}) = (\mathrm{ev}(\text{查询}),\ \mathrm{ch}(c_{\mathrm{old}}) + \mathrm{ch}(c_{\mathrm{next}})),$$

其中子树值的第一分量为零。剩余计数逐项相加，再加两次加法与一次线性嵌入，共三个组合节点：$r(\text{主}) = r(\text{查询}) + c_{\mathrm{old}} + c_{\mathrm{next}} + 3$。已付迹长度：所有者无关安装的迹长等于剩余计数，查询自身的已付迹长度等于 $r(\text{查询})$，代入即得。两棵子树的执行长度等于各自的剩余计数（命题 3.7(2)）。旧库存与新付事件的保留由并列种子的嵌入给出；旧、新作用者分量之和是本次源操作的状态点，由完整作用者载体的读出读回。

(3) 子树环境。在被供给的发生处，子树环境由该环境中原绑定的读出定义；新生帧的环境取实际更新后的环境（定理 9.5(5)），子树环境随之由同一绑定读出。标量与配对库存的保留与 (2) 同理；运行时初始节点与每拍输入是运行时的定义式。■

形式对应。`ACT/Inventory/Pair/Residual/Joint/Past/NextObservation/Request/Acted/{Source,Consumer}.lean`；`DJ/{Joint,Branch}/Primary/{Source,Consumer}.lean`；`DJ/Branch/JointQuery/Actor/{Source,Embedding,Action/Source,Operation/Source,Operation/Consumer}.lean` 与 `Actor/Operation/Query/Inventory/{Language,Source,Consumer,Action/Source,Action/Consumer,Live/Source,Live/Consumer}.lean`；默认后继 `DJ/Next/{Source,Consumer}.lean`。提交 AE。

S5.2 定理 8.15：真实总值、逐回执 decoder 与 Born

同次实际材料。原动作读取器在准确发生 o 给出 Raw (ρ, e)，原 result 给出已付状态 $\mathrm{const}(p)$、完整 Trace 及实际 $p = \mathrm{ev}_{\rho} e$。原 material M 保存这些对象与实际 afterEnvironment 的差 ε；新请求 $N(M)$ 是 $e \ominus \mathrm{const}(p)$，其登记环境为 $\rho + \varepsilon$。所以

$$\mathrm{ev}_{\rho+\varepsilon} N(M) = \mathrm{ev}_{\rho+\varepsilon} e - p =: n, \qquad p + n = \mathrm{ev}_{\rho+\varepsilon} e.$$

第一个等式逐构造子展开减法；第二个等式只使用加群消去。p 与 n 分别是旧完整查询的已付值和原 residual 的实际完成值，未输入更新后的目标值。

实际程序的改动。保留原 lowProgramme 的 source、reader、calculationReader、effect/inverse、whole 及两库存回调，仅把其 `nextEnvironmentReadAt` 改为 $\mathrm{write}(p + \nu(o'))$。这里 ν 对实际 native event 消去：付款支取零，结算支取该结算的 value。新程序沿原帧运行，原 Type 回执选中的结算事件 o_B 的 $\nu(o_B)$ 恰为 n。展开原 settled 编译的 nextBorn 环境字段，得

$$\mathrm{env}(\mathrm{nextBorn}) = \mathrm{write}(p + \nu(o_B)) = \mathrm{write}(p + n).$$

付款支的原编译目标是 mathNext，不调用这份出生环境；它保留原登记输入、状态与 Trace 再走一步。因此 decoder 的字段存在于程序中，实际写回由原结算分支执行。上述总值恒等只使用同一 actionReader 给出的 ρ, e 与其已付端点。在 S2.3 的实际 Contextual/Incoming 源族中，另已生成的 environment square 将原 queryReader 环境与这份 actualMaterial 对齐。

逐包再生。设原 Factory 的 j 包为 P_j。其 receiver 的两个库存、原计算读取器和原语言均保留。上述回执给出实际 Born B_j；下一包 P_{j+1} 由该 Born 的完整材料及其登记表达式形成新 seed。`nextCfg` 对这个实际下一

包重读其原 actionResult，取 p_{j+1}，对其 residual 的实际事件取 ν_{j+1}，并安装 $\mathrm{write}_{j+1}(p_{j+1}+\nu_{j+1})$。将上一段用于 P_{j+1} 得

$$\rho_{B_{j+1}} = \mathrm{write}_{j+1}(p_{j+1}+n_{j+1}).$$

此式没有沿用 p_j。两代消费者用原完整 next packet 等式运输原语言，恢复 B_j，再得到 `secondNextCfg` 的相同构造；对 j 归纳给出每份回执的 fresh decoder 与两种库存保留。■

零效应对照。原配置查询为 $\mathrm{const}(1,0)$ 时，$p=(1,0)$、$n=0$，实际 Born 写回 $(1,0)$。下一配对代的原已付值为 $((1,0),(0,0))$，仍不为零；重新生成的解码器保存它。这个实际控制说明 native effect 不能替代 total value。

固定来源。MV3 的 `ACT/Receipt/Configured/Writeback/{Source,Consumer}.lean`、`Writeback/Successor/{Source,Consumer}.lean` 的 `old_paid_value`、`native_value`、`updated_total`、`born_environment`、`successorPacket`、`second_decoder`、`second_writeback`；原 comb 的 `Feedback/ClozelOriginalKConfiguredPairWriteback.lean`、`Feedback/ClozelOriginalKSourceSuccessor.lean` 及其 Verification 直接消费者。

S5.3 定理 9.10：同一运行的完整共尾片段

原运行的两个阶段。实际 Stage 是依赖和：admission 保存级别、完整 datum、configuration 与 language；continuation 保存这些对象和原 paid Frame。原 advance 在 admission 分支把该包的实际准入送入 continuation；continuation 的 paid 分支只取 mathNext，级别与原包材料不变；settled 分支取原 nextBorn，把其 registered expression 接入下一 seed，再回到 admission。库存 preserve 逐层装入旧标量、旧配对及本次实际生成的库存，actual actionReader 仍取原 calculationReader 若有，否则原 reader。整个 Stage 是一份源生成函数，而非外部日程。

到下一包的 Type 数据。对任意 tick t，消去其 actual Stage。admission 返回距离零、原包与帧等式。continuation 调用 S1.3 的同一递归帧回执；它给出准确距离 B、全部 B 次付款与原 settled frame。原 advance 在其下一拍就是 admission，故距离 $B+1$。把每个中间 paid Frame 与原 Stage 等式存入片段；没有压缩原时钟或另起运行。严格秩取（库存大小，因果深度加一）：admission/settled 增长库存，paid 保持库存并增加深度，故 source erasure 在原 ticks 上单射。

停点构造与覆盖。令 $d(t)$ 为上述实际距离，$t_0=d(0)$，$t_{j+1}=t_j+1+d(t_j+1)$。距离数据给出两个端点处原 Stage 的 admission 等式；每个 j 的 historyAt 是原 tick t_j+h 的完整帧，$0\le h\le \ell_j$，$\ell_j=1+d(t_j+1)$。于是

$$t_j+\ell_j=t_{j+1},\qquad t_{j+1}\ge t_j+1,\qquad t_j\ge j.$$

对任意 t 取 $j=t$ 即得 $t\le t_j$，故严格共尾；若需覆盖片段，取第一个超过 t 的停点，前一个停点到它的上述 historyAt 包含 t。相邻包的源数据等式由 actual admission、nativeAfterAdmission 和 bornAfterNative 逐字段组成；级别加一只出现在包的准入转换，不替代 actual tick。

完整原词上的推进。令 W_t 为原运行的完整场词，原点作用满足 $A_{\mathcal{R}}W_t=W_{t+1}$。对 h 归纳得到 $A^h_{\mathcal{R}}W_t=W_{t+h}$；在 $h=\ell_j$ 使用索引等式便得定理 9.10(3)。在原自由载体上由线性延拓，原完整 source word 与场词、共尾点单射及词—状态对偶都沿同一 source span 交换。片段中的付款 trace 与准入 stock 从未从这份载体删除。■

固定来源。MV3 的 `F/Admission.lean`、`F/SourceFamily/{Runtime,Activation/Source,Activation/Stage,Activation/Admissions}.lean`，其中 `F/` 的完整展开为正文附录 D.3；完整词消费者 `Operations/`

`Inquiry/Cofinal/{Source,Consumer}.lean` 的 source-span、strict、cofinal、cover 与 field/point 交换声明。

S5.4 定理 9.12：完整原词作用与全部未来模型

每个真实包的原语言。第 j 个停点的 installed Raw、query incidence 与 full packet equality 来自 S3.3 和 S5.3。其完整词模 $\mathcal{W}_j$ 的原生读出为

$$d_j(u) = (\mathrm{ev}_{\rho_j} u, \mathrm{eff}_{\rho_j,\varepsilon_j} u).$$

把 $\mathcal{W}_j$ 的表达式逐节点作原配对提升，再按原下一包的语言等式运输，得到原 `sourceWordAction` S_j。附录 A.2 在每个表达式上给出配对求值等式，线性延拓得

$$\mathrm{ev}_{\rho_{j+1}^{\mathrm{receiver}}}(S_j u) = d_j(u).$$

接收器环境的 old/increment 两分量由该 receiver 的原注册环境读回；完整 next packet 等式认回 grade、seed、Frame 与 language。这证明的是实际接收器方块，不推导 $\rho_{j+1} = \rho_j$。更新环境 $\rho_j + \varepsilon_j$ 仍取原材料的真实 increment，阶段完成的 update map 由旧读出加 effect 生成。

库存与有限支撑。原 scalar relation stock 的成员经配对提升进入 born relation stock；原 pair stock 和各 paid 前缀在 mathNext 保持、在 nextBorn 经 preserve 嵌入；原 seed 的嵌入给出 source action 的库存成员证明。命题的“词属于库存”指其在实际关系库存的成员或由实际库存生成的词，未给任意词加上这个成员证明。令 $\mathcal{W} = \bigoplus_j \mathcal{W}_j$；对有限支撑 w，定义

$$S(w) = \sum_{j \in \mathrm{supp}(w)} \mathrm{in}_{j+1}(S_j w_j), \qquad D_j(w) = d_j(w_j).$$

支撑经 $j \mapsto j+1$ 仍有限，故 S 定义良好且线性；原逐包运输使此 S 正是 original common action。所有 site maps 组成 jointObserver。有限前缀 $0, \dots, b$ 的 evaluator 以删除末位相容，生成原联合完成载体；其纤维由每个 site 的 lifted difference 落入该 site 原 pair relation range 准确刻画。

全部未来词与最大核。允许字母集合是 $\{*\}$，该字母的作用是上述 S；令 $S_{[]} = \mathrm{id}$、$S_{a*} = S \circ S_a$。定义

$$K_{\mathrm{fut}} = \bigcap_{a \in \{*\}^{<\omega}} \bigcap_j \ker(D_j \circ S_a).$$

它是子模，且 $S(K_{\mathrm{fut}}) \subseteq K_{\mathrm{fut}}$：对 $w \in K_{\mathrm{fut}}$、任意 a, j，$D_j(S_a S w)$ 是多加一个原字母后的读数，故为零。商映射 q_{fut} 因而给出唯一模型作用 $\bar{S}$，满足

$$\bar{S} q_{\mathrm{fut}} = q_{\mathrm{fut}} S, \qquad \bar{D}_{j,a} q_{\mathrm{fut}} = D_j S_a.$$

任意 u, v 的商类相等当且仅当 $u - v \in K_{\mathrm{fut}}$，逐个核展开就是全部允许未来词、全部 site 的读数相等。若 L 当前不可见且 $S(L) \subseteq L$，对词长归纳得 $S_a L \subseteq L$，再应用当前不可见得 $D_j S_a L = 0$；故 $L \subseteq K_{\mathrm{fut}}$。这给出最大性，未把一组指定未来采样当作全部未来。

接回原完成。空词给出 $K_{\mathrm{fut}} \subseteq \bigcap_j \ker D_j$，故 $q_{\mathrm{fut}}(w)$ 到原 jointCompletion 的 source map $q_{\mathrm{now}}(w)$ 良定义；与 $\bar{S}$ 复合所得正是 $q_{\mathrm{now}}(Sw)$。S5.3 的实际 source span 把共尾包的词作用接到全部原 ticks 上的完整 State 作用。全部未来 Model、原 completion update、receiver square、actual inventories 与 literal next 在同一条原源链上。■

固定来源。MV3 的 `Operations/Inquiry/Cofinal/Native/Installed/Cofinal/Tower/{Source,Consumer,Action/Source,Action/Consumer,Action/Common/Source,Action/Common/Consumer}.lean`，原最大核证明 `Operations/Observed/Words/{Kernel,Model}.lean`（均在 `GenericFoundation/` 下）。

S5.5 §10.1：原 comb 的实际值反馈与完整 R

原计算的输入与安装。原观察提供非平凡零点坐标 ζ、由该原观察读出的 Xi 精确阶 m 及原 comb q_0。原算子的 resolvent 参数为 $z=\zeta/2$；原右半证明 $\Re\zeta>1/2$ 给出 $\Re z>1/4$。定义 $e_0=x$、$e_{k+1}=\mathrm{linear}_R(e_k)$。按 k 归纳，

$$\mathrm{ev}_\rho e_k=R^k(\rho(x)),\qquad r(e_k)=k+1.$$

在 $\rho(x)=q_0$ 上执行得到实际 $v=R^mq_0$ 与长度 $m+1$ 的完整 Trace。原 named facade 在 emitter 之前把这份 result law 安装到原 authority source 的同一次发生；计算面由该发生读回 z,m,q_0,e_m 及其 actual result。原 projection coface 的余项保留完整旧 ledger/emitter，所以原物理 next 与原 lower authority 没有被这笔计算替换。关于原 v 的 physicality 来自原 Xi 阶消费者，不是给定完成值。

由 actual value 生成增量。取上段实际 v，定义 $\rho^+=\mathrm{constEnv}(v)$、$\delta=\rho^+-\rho$。已付 material 保存原 e_m 与 $\mathrm{const}(v)$，请求值为

$$\mathrm{ev}_{\rho+\delta}(e_m\ominus\mathrm{const}(v))=R^mv-v.$$

其表达式预算严格为正；原 lowProgramme 的配对提升以 decoder $(a,b)\mapsto a+b$ 写回完整更新值。S1.3 的 Type 回执和 S5.2 给出 actual nextBorn 环境 $\mathrm{constEnv}(R^mv)$。这里 δ 专指由计算结果生成的 comb 增量；一般 actualMaterial 中的 ε 是 actual afterEnvironment 减原环境，两者只有在这个配置的环境等式证明后相等。

完整 R 的三个原语步。令 R_z 为原 ambient resolvent，P 为原闭子空间的正交投影。配对输入为 $(v,0)$，原程序是先作 R_z，后作 split：变量读取 $\to(v,0)$，线性 $R_z\to(u,0)$，拆分 $\to(p,r)$，其中 $u=R_zv$、$p=Pu$、$r=(1-P)u$。三个节点各付一步，完整 Trace 保留第二节点 $(u,0)$；$p+r=u$ 是原两正交投影之和为恒等的等式。物理 decoder 取 p，完整 stock 保存 r 及整个程序。

原 Mellin 定价。在原参数范围 $\Re z>1/4$、$1/4<\Re w<1/2$，记 L_w 为原 completed ambient Mellin 泛函。原 resolvent 的 Bochner 积分与连续线性 L_w 交换；原 dilation 法则把其 integrand 读为权重乘字符再乘 $L_w(v)$。乘积核为 $\exp((1/2-z-w)h)$，其实际衰减满足 $\Re(1/2-z-w)<0$，所以

$$L_w(R_zv)=-\int_0^\infty e^{(1/2-z-w)h}\,dh\;L_w(v)=-\frac{L_w(v)}{z+w-1/2}.$$

原 Gamma–Mellin 桥在上述范围以 $\Gamma_{\mathbb{R}}(2w)\neq0$ 消去归一化，给出 physical heat $H_w(v)=2L_w(v)$；$M_w(r)=L_w(r)$。应用 $p+r=R_zv$ 的线性读出并乘二，便得

$$H_w(p)+2M_w(r)=-\frac{H_w(v)}{z+w-1/2}.$$

因此定价消费完整 R_zv 和其原拆分，完整余项参与等式。

原下一查询。真实 configuration 的 calculationReader 装入同一三步程序，Frame/stock 恢复 actual Born 和下一 Raw。下一配对输入由 literal physical Born 的 p 产生，实际结果为 $(PR_zp,(1-P)R_zp)$，长度仍为三。在同一登记环境下该结果的 residual 值为零，S5.2 仍把完整 oldPaid 写回，而非零；下一 successor

packet 再从自己的 actual result 生成 decoder。原 programme、完整 trace、两库存和全词 Model 同时保留。■

固定来源。计算面来源 MV2，实际反馈与完整 R 消费者来源 MV3。`NoIslandNoMagic/CanonicalRiemann/GeneralizedDual/CoPoissonEnergy/Muntz/Cokernel/CenteredGram/JointAction/Runtime/` 下 `Material/Calculation/ClozelOriginalKCombSourceCalculation.lean`、`Root/ClozelPairedOmegaEffectRowRoot.lean`、`Facade/ClozelPairedOmegaEffectRuntimeFacade.lean`、`Feedback/ClozelOriginalK{ReceiptSource,ReceiptFeedback,PhysicalPricing,PhysicalAction,PhysicalFeedback,RegisteredSourceFeed,ConfiguredPairWriteback,SourceSuccessor}.lean` 及原 Verification 直接消费者。

S5.6 §10.1：37 原语到逐 tick 身体与有限付款历史

原输入与 Type 0 实装。固定原 37 项原语输入 I。原有限 programme 依次翻译、修复、资源响应与化学交换，`generated_input(I)` 返回这次实际计算的 occurrence；成功分支由原正向完成证明排除失败，而未从外部输入一个成功身体。依赖类型 $\mathsf{InputBodyAt}(I)$ 按原结果消去：缺少 programme 时存精确缺失处置；保留失败结果时存原失败；响应成功并满足原 profile 时存从该真实 disposition 产生的 body；profile 失败时存其原 residual。固定源实例的原正向证明认回其实际成功分支，因而身体中 DNA、full coding、蛋白、核/电子场、resource account、CP focus 与开责任、完整库存及 written history 都由原值字段生成。

原 Native current 为 (phase, I, B)，$B : \mathsf{InputBodyAt}(I)$。原更新函数在该 body 上计算 actual body event 与 bodyWrite；其 target 定义为

$$\mathrm{next}(\mathrm{phase}, I, B) = (\mathrm{phase}+1, I, \mathrm{bodyNext}_I(B)).$$

原 Write 同时保存 actual body 与其等于该 next-body 的方程，原 Root 编译读取这个 Write。故 literal next 的 body 就是本 tick 的实际 written body。对 tick 归纳，输入始终是同一 I，运行中的 body 是从原 B 反复应用同一 bodyNext 的结果；原高层证据作为 body 的类型索引运输，不向根再提供一份未来身体。原 Delivery 的 readNativeBody/readNativeWrite/readWrittenNativeBody 与这个 Root.current、emitted Write 和 target 逐字段相等。

完整平滑与时间精化。在实际 PostState 上，电子 occupied 矩阵与核姿态取原半步 Cayley 更新，核/电子 full raw energy 包含原全部核项、电子 trace 与二体 quartic 项。每个 Gaussian nuclear integral 的 stored/local 四种项可在积分下微分，有限 nuclei 与矩阵项求和保微分；分离姿态处全部核对距离非零。因此该实际 state 的 full smoothness 由原场产生，不输入某个未来 force。令原 ready candidate 的 price 为 $\pi(t)$。半步坐标趋于原 occupied、完整 force 连续、核位移趋于零，故足够小时间保持核分离；完整末端 occupied 与 full raw energy 连续，所以

$$\pi(0) = 0, \qquad \lim_{t\to 0} \pi(t) = 0.$$

原精化在保持实际 fields、stock、account 的同时把 timestep 变为 $t_k = t_0\, 2^{-k}$。ready 从某个 k 起恒成立。

源生成的有限 disposition。若 reserve $A > 0$，由连续性取 k 使 ready 且 $\pi(t_k) < A$，所以弱 guard ready 且 $\pi(t_k) \le A$ 有解。原构造以 Nat.find 取首个满足 guard 的 k_*，得到包含 k_*、其实际事件、完整 afterState、更新库存/账及结果等式的 Type disposition。每个 $k < k_*$ 都未付，原 body event 在该分支等于原 refineState；归纳故实际 k 次 bodyNext 的状态就是上述精化状态。

若 $A = 0$，先取首个 ready 的 k_*，存在性仍来自小时间分离。实际 guard 随后比较 $\pi(t_{k_*}) \le 0$：成立则生成 paid；不成立则生成准确 energyShortage，并保存首个 ready 与正 price 的证据。$k < k_*$ 都非 ready，故仍是未付精化。两支均为有限、源自产的 Type disposition；没有声称零 reserve 总能付款。

在原 Root 上逐 tick 兑现。对从任意实际 body 出发的步数 k 归纳，原 bodySteps 的完整 body 与同一 sealed runtime 的 k 次 literal next body 异构相等；输入 I 未变使类型运输有唯一原索引。代入 k_*，上段选中的实际 bodyEvent 是原 Delivery 在实际 tick k_* 读出的 NativeWrite，written body 是原 bodyNext，原下一 tick 读取的 body 仍是它。片段包含此前每个真实未付事件的原编译与身体字段。paid 支保存原电荷／电子数、粒子与场以及 $E + A$ 的完整守恒并推进实际 elapsed；未付精化的 elapsed 按原律保留。于是固定 37 原语从每个实际 tick 生成有限 paid history（$A > 0$）或准确零 reserve disposition（$A = 0$），由同一个原 Root 连续运行。■

固定来源。MV3，`NoIslandNoMagic/Consciousness/Immortality/Embodied/Molecule/CPS1Personalized2025/` 下 `Source/NativeRegistration/{Source,Programme,Registration,Positive,BodyPositive}`、`Root/{NativeCurrent,Base,Installation}`、`Delivery/Runtime`、`Runtime/{RegisteredBody,BodyPaid,BodyFunction}`；`Lean/DomainProofs/CPS1ResourceExecution/CPS1MaterialIncidence/` 下 `NativeRoot{FromInput,Data,MaterialData,NextLaw}`、`Native{RawRegularity,FullSmooth,BodyPaidSource,BodyFunction}`（以上均 `.lean`）。临床标签只为原 source row 的标签，本节未从它推导临床效果，也未把后续 governed action 替换为这里的 original Native.next。

S6 选集编号与证明位置

下表把首发选集的主张编号对应到正文结果与书面证明位置。选集编号与公开代码选集的首发映射（H0mework `docs/first-release-map.json`）中的 `core.Cn` 一致，供逐条复核。

选集编号	正文结果	书面证明	源提交
core.C1	§6.1 命题 6.1	正文	H（base 回归修复）
core.C2	§6.4 定义 6.6、定理 6.7、注 6.8	正文	H
core.C3	§2.1 定义 2.1、命题 2.2	正文	H
core.C4	§8.4 运行时段	正文	H
core.C5	§8.4 命题 8.9、表 4、图 5	正文；附录 C.3	H
core.C6	§8.4 命题 8.10	正文	H
core.C7	§3.1–§3.3 定义 3.1–3.2、命题 3.3–3.6	正文	H
core.C8	§4 定理 4.4、4.6、4.7、命题 4.8	正文；附录 A–B	H
core.C9	§2.2–§2.4 命题 2.3、定义 2.4、命题 2.5	正文；附录 A.1	H
core.C10	§5 定义 5.1、命题 5.2、定理 5.3	正文	H
core.C11	§7.1–§7.3 命题 7.1、定义 7.2、命题 7.3–7.5	正文；附录 C.6	H
core.C12	§6.2–§6.3 定义 6.2–6.3、命题 6.4–6.5、表 2	正文；附录 C.5	H
core.C13	§8.1–§8.3 定义 8.1、命题 8.2–8.3、定理 8.4、定义 8.5、定理 8.6–8.7、命题 8.8	正文；附录 C.1–C.4	H
core.C22	§6.5 定义 6.9、定理 6.10	S1	AB
core.C23	§3.4 命题 3.7、定义 3.8、定理 3.9；§6.5 命题 6.11；§10.1	正文（完整）	AB

选集编号	正文结果	书面证明	源提交
core.C24	§8.5 定义 8.11、定理 8.12、例 8.13、图 6	S2.1	AB
core.C25	§7.3 命题 7.6	正文（完整）	AB
core.C26	§7.3 定理 7.7(1)(2)	S2.2	AC
core.C27	§7.3 定理 7.7(3)	S2.2	AC
core.C28	§9.1 定义 9.1、定理 9.2	S3.1	AD
core.C29	§9.2 定理 9.3	S3.2	AD
core.C30	§9.3 定义 9.4、定理 9.5	S4.1	AD
core.C31	§9.3 定理 9.6	S4.2	AE
core.C32	§9.3 定理 9.7	S4.3	AE
core.C33	§9.4 定理 9.8(1)	S5.1	AE
core.C34	§9.4 定理 9.8(2)(3)	S5.1	AE

第二版四组收稿沿用各自的固定来源；本表补入其逐条书面位置。CPS1 为本版实装实例，未增加四组 intake。

选集主张／实例	正文结果	书面证明	来源
C35–C36	§8.5 同次环境与两种费用；定理 8.14	S2.3	PR、H0-S
C37–C38	定理 8.14	S2.3	PR、MF
C39	定理 6.12；§7.3	S1.3	MV2
C40	定理 9.9	S4.4	MV2
C41	§10.1 原 comb 计算	S5.5	MV2
C42	定理 8.15；§10.1 实际 comb 反馈	S5.2、S5.5	MV3
C43	定理 9.10；定理 6.12 的原编译消费者	S5.3	MV3
C44	定理 9.11	S3.3	MV3
C45	定理 9.12	S5.4	MV3
CPS1 实装	§10.1 37 原语／逐 tick 身体／有限付款历史	S5.6	MV3